



FACULTAD DE
ESTUDIOS PROFESIONALES
ZONA MEDIA



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS PROFESIONALES ZONA MEDIA

Práctica 15

Uso de protocolo Bluetooth LE en la
Raspberry Pi Pico W para enviar y recibir
datos inalámbricamente

Carrera:

Ingeniería Mecatrónica – Cuarto Semestre

Alumno:

Saúl Martínez Almazán

Clave:

Docente:

Ing. Jesús Padrón

Fecha:

25 de Abril de 2026

Índice

1. Introducción	2
1.1. Objetivos	2
1.2. Trabajo Previo	2
1.2.1. Bluetooth LE vs. Bluetooth Clásico	2
1.2.2. Arquitectura GATT	3
1.2.3. BTstack	3
2. Desarrollo de la Práctica	3
2.1. Estructura del Proyecto	3
2.2. Configuración del Entorno de Desarrollo	4
2.3. Descripción de los Archivos de Código	4
2.3.1. Archivo <code>temp_sensor.gatt</code> – Definición del servicio BLE	4
2.3.2. Archivo <code>btstack_config.h</code> – Configuración de BTstack	5
2.3.3. Archivo <code>server_common.c</code> – Lógica del servidor	5
2.3.4. Archivo <code>server.c</code> – Punto de entrada del servidor	5
2.3.5. Archivo <code>client.c</code> – Punto de entrada del cliente	6
2.4. Proceso de Compilación	6
2.5. Montaje y Prueba	6
2.6. Complicaciones y Soluciones	8
3. Preguntas de Repaso	8
4. Conclusiones	12
Anexos	13
Evidencia Fotográfica	17

1. Introducción

En la presente práctica se implementa una comunicación inalámbrica de corto alcance entre dos microcontroladores **Raspberry Pi Pico W** utilizando el protocolo **Bluetooth Low Energy (BLE)**. El objetivo central es configurar una Pico W como **servidor BLE** que lee la temperatura interna del chip a través de su ADC y la transmite periódicamente, mientras que una segunda Pico W actúa como **cliente BLE** que escanea, se conecta y recibe dichos datos, mostrándolos en el monitor serial de la computadora.

La relevancia de este tipo de sistemas en la industria embebida es considerable. El protocolo Bluetooth LE, al operar con un consumo energético muy reducido comparado con WiFi o Bluetooth Clásico, es la tecnología preferida en aplicaciones de monitoreo de sensores, dispositivos portátiles (wearables), redes IoT industriales y sistemas de salud. Los conceptos de perfil GATT, servicios y características BLE que se abordan en esta práctica son exactamente los mismos que se encuentran en dispositivos comerciales como monitores de glucosa, pulsómetros y sensores ambientales inteligentes.

La implementación se realiza en lenguaje C mediante el Pico SDK, haciendo uso de la biblioteca **BTstack** de BlueKitchen, que provee una implementación completa de la pila Bluetooth optimizada para sistemas embebidos de recursos limitados.

1.1. Objetivos

- Comprender los conceptos básicos del protocolo Bluetooth LE y su arquitectura GATT.
- Configurar una Raspberry Pi Pico W como servidor BLE que expone el sensor de temperatura interno.
- Configurar una segunda Raspberry Pi Pico W como cliente BLE capaz de descubrir, conectar y recibir notificaciones del servidor.
- Familiarizarse con la biblioteca BTstack y su modelo de programación basado en máquinas de estados y eventos.

1.2. Trabajo Previo

1.2.1. Bluetooth LE vs. Bluetooth Clásico

El estándar Bluetooth nació en 1994 en Ericsson como una alternativa inalámbrica de corto alcance a los cables entre periféricos. Con el tiempo se desarrollaron dos variantes principales:

Bluetooth Clásico (BR/EDR) opera sobre 79 canales en la banda de 2.4 GHz con tasas de transferencia de hasta 3 Mb/s. Está orientado a aplicaciones que requieren transmisión continua de datos, como audio estéreo (A2DP) o transferencia de archivos.

Bluetooth LE opera sobre 40 canales (37 de datos y 3 de anuncio) con tasas de hasta 2 Mb/s, pero con un consumo de energía drásticamente menor (típicamente

$\leq 10\text{mW}$). Está diseñado para comunicaciones intermitentes y es ideal para sensores, beacons y dispositivos alimentados por baterías pequeñas.

1.2.2. Arquitectura GATT

El **Generic Attribute Profile (GATT)** define cómo los dispositivos BLE organizan e intercambian datos. La estructura jerárquica es la siguiente:

- **Perfil:** conjunto de servicios relacionados que describen un caso de uso completo.
- **Servicio:** agrupación lógica de características. En esta práctica se usa el servicio estándar **Environmental Sensing**.
- **Característica:** unidad básica de datos con un UUID, valor, y descriptores que especifican propiedades como lectura, escritura o notificación.

El servidor BLE mantiene una *tabla de atributos* (ATT) con todos sus servicios y características. El cliente puede leerlas directamente o suscribirse a notificaciones para recibirlas automáticamente cuando cambian.

1.2.3. BTstack

BTstack es una implementación de la pila Bluetooth desarrollada por BlueKitchen, diseñada específicamente para sistemas embebidos de 8 a 32 bits. Sus características clave son:

- **Diseño de un solo hilo:** utiliza un único bucle de ejecución (run loop) con fuentes de datos y temporizadores, sin requerir multitarea.
- **No bloqueante:** todas las operaciones Bluetooth son asíncronas; los resultados se entregan como eventos a través de manejadores de paquetes registrados.
- **Memoria estáticamente limitada:** el número máximo de conexiones y servicios se configura en tiempo de compilación mediante `btstack_config.h`.

2. Desarrollo de la Práctica

2.1. Estructura del Proyecto

El proyecto se compone de dos ejecutables independientes que se convierten en archivos `.uf2` para flashear en cada Raspberry Pi Pico W:

Cuadro 1: Archivos del proyecto y su función

Archivo	Ejecutable	Función
server.c	3*picow_ble_temp_sensin	Implementa Bluetooth, ADC y el timer del servidor
server_common.c		Lógica de anuncios, callbacks ATT y lectura de temperatura
temp_sensor.gatt		Define los servicios y características BLE del servidor
client.c	picow_ble_temp_reader	Encuentra, conecta y recibe notificaciones de temperatura
btstack_config.h	Ambos	Configuración en tiempo de compilación de BTstack
CMakeLists.txt	Ambos	Configuración del sistema de compilación

2.2. Configuración del Entorno de Desarrollo

Para compilar correctamente el proyecto se requieren las siguientes herramientas instaladas en Windows:

1. **Pico SDK** clonado en C:/pico-sdk con todos los submódulos inicializados:

```

1 git clone https://github.com/raspberrypi/pico-sdk.git C:/
  pico-sdk
2 cd C:/pico-sdk
3 git submodule update --init --recursive

```

2. **ARM GNU Toolchain** (arm-none-eabi-gcc) extraído en C:/arm-toolchain y su carpeta bin/ agregada al PATH del sistema.
3. **CMake** instalado mediante `winget install Kitware.CMake`.
4. **Ninja** instalado mediante `winget install Ninja-build.Ninja`.

2.3. Descripción de los Archivos de Código

2.3.1. Archivo temp_sensor.gatt – Definición del servicio BLE

Este archivo define la tabla de atributos del servidor. Declara tres servicios: el servicio GAP con el nombre del dispositivo, el servicio GATT estándar, y el servicio de *Environmental Sensing* con la característica de temperatura configurada con permisos de lectura, notificación e indicación dinámica.

```

1 PRIMARY_SERVICE , GAP_SERVICE
2 CHARACTERISTIC , GAP_DEVICE_NAME , READ , "picow temp"
3
4 PRIMARY_SERVICE , GATT_SERVICE
5 CHARACTERISTIC , GATT_DATABASE_HASH , READ ,
6
7 PRIMARY_SERVICE , ORG_BLUETOOTH_SERVICE_ENVIRONMENTAL_SENSING
8 CHARACTERISTIC , ORG_BLUETOOTH_CHARACTERISTIC_TEMPERATURE ,
9     READ | NOTIFY | INDICATE | DYNAMIC ,

```

2.3.2. Archivo `btstack_config.h` – Configuración de BTstack

Contiene las directivas de preprocesador que configuran la pila Bluetooth en tiempo de compilación. Define los roles habilitados (periférico para el servidor, central para el cliente), los tamaños de buffer ACL, el número máximo de conexiones y las capacidades de seguridad.

La directiva más importante es la condicional `RUNNING_AS_CLIENT`, que activa el rol central y el soporte para clientes GATT únicamente cuando se compila el ejecutable del cliente, permitiendo que ambos proyectos compartan el mismo archivo de configuración.

2.3.3. Archivo `server_common.c` – Lógica del servidor

Implementa tres elementos fundamentales:

- **packet_handler:** manejador de eventos HCI que configura los parámetros de anuncio BLE cuando el stack está activo y gestiona las desconexiones.
- **att_read_callback / att_write_callback:** callbacks invocados por BTstack cuando el cliente lee o escribe atributos. El callback de escritura detecta cuándo el cliente activa las notificaciones.
- **poll_temp:** lee el canal ADC 4 (sensor de temperatura interno), escala la lectura de 12 a 16 bits y aplica la fórmula de conversión $T = 27 - \frac{V_{be} - 0.706}{0.001721}$ para obtener grados Celsius.

2.3.4. Archivo `server.c` – Punto de entrada del servidor

Inicializa el driver CYW43 (que habilita el módulo Bluetooth), el ADC con el sensor de temperatura, las capas L2CAP y SM de BTstack, el servidor ATT con los callbacks registrados, y configura un timer de latido (*heartbeat*) que cada 10 ciclos (1 segundo) llama a `poll_temp()` y envía la nueva temperatura mediante notificación si el cliente está suscrito.

2.3.5. Archivo `client.c` – Punto de entrada del cliente

Implementa una máquina de estados con ocho estados que guía el proceso completo de conexión BLE:

Cuadro 2: Estados de la máquina de estados del cliente BLE

Estado	Descripción
TC_OFF	Stack apagado o no inicializado
TC_W4_SCAN_RESULT	Escaneando dispositivos BLE cercanos
TC_W4_CONNECT	Conectando al servidor encontrado
TC_W4_SERVICE_RESULT	Descubriendo el servicio Environmental Sensing
TC_W4_CHARACTERISTIC_RESULT	Descubriendo la característica de temperatura
TC_W4_ENABLE_NOTIFICATIONS_COMPLETE	Activando notificaciones en el servidor
TC_W4_READY	Listo, recibiendo notificaciones de temperatura

2.4. Proceso de Compilación

Con todos los archivos en la carpeta del proyecto, la compilación se realiza desde la terminal:

```

1 cd "C:\Users\soysa\OneDrive\Documentos\Practica 15"
2 mkdir build
3 cd build
4 cmake .. -G "Ninja" -DPICO_SDK_PATH=C:/pico-sdk -DPICO_BOARD=
  pico_w ^
5 -DCMAKE_C_COMPILER=C:/arm-toolchain/.../bin/arm-none-eabi-
  gcc.exe ^
6 -DCMAKE_CXX_COMPILER=C:/arm-toolchain/.../bin/arm-none-eabi-
  g++.exe ^
7 -DCMAKE_ASM_COMPILER=C:/arm-toolchain/.../bin/arm-none-eabi-
  gcc.exe
8 ninja

```

Al finalizar correctamente se generan en la carpeta `build/` los archivos:

- `picow_ble_temp_sensor.uf2` → se flashea en la RPi Pico W 1 (servidor)
- `picow_ble_temp_reader.uf2` → se flashea en la RPi Pico W 2 (cliente)

2.5. Montaje y Prueba

Para flashear cada Raspberry Pi Pico W se mantiene presionado el botón **BOOTSEL** mientras se conecta el cable USB. La placa aparece como unidad de almacenamiento RPI-RP2 y se arrastra el archivo `.uf2` correspondiente.

La RPi Pico W que actúa como cliente se conecta a la computadora para visualizar la salida en el monitor serial (115200 baudios). El servidor puede alimentarse desde cualquier fuente USB. Al funcionar correctamente, el monitor serial muestra:

```
1  ---- Opened the serial port COM14 ----
2  BTstack up and running on 28:CD:C1:0C:6D:4E.
3  Connecting to device with addr 28:CD:C1:0C:6D:4E.
4  read temp 31.25 degc
5  read temp 30.78 degc
6  read temp 31.72 degc
7  read temp 30.32 degc
8  Disconnected 28:CD:C1:0C:6D:4E
9  Connecting to device with addr 28:CD:C1:0C:6D:4E.
10 read temp 30.32 degc
```

Las temperaturas entre 30 y 32°C corresponden a la temperatura interna del chip RP2040, que es normal durante operación. La reconexión automática funciona correctamente al perder y recuperar la conexión BLE.

2.6. Complicaciones y Soluciones

Cuadro 3: Errores encontrados durante el desarrollo y sus soluciones

Error Encontrado	Solución Aplicada
CMake no reconocido en la terminal a pesar de estar instalado.	Se instaló CMake con <code>winget install Kitware.CMake</code> y se cerró/reabrió la terminal para que el PATH tomara efecto.
CMake usaba el GCC de MinGW en lugar del compilador ARM, generando error <code>unrecognized option '-mthumb'</code> .	Se especificaron explícitamente las rutas de <code>CMAKE_C_COMPILER</code> , <code>CMAKE_CXX_COMPILER</code> y <code>CMAKE_ASM_COMPILER</code> apuntando al <code>arm-none-eabi-gcc</code> del toolchain ARM.
Error <code>Unknown CMake command "pico_btstack_make_gatt_header"</code> por submódulos no inicializados.	Se ejecutó <code>git submodule update -init -recursive</code> en la carpeta del SDK. El submódulo <code>btstack</code> falló inicialmente y fue clonado manualmente con <code>git clone</code> .
La ruta del proyecto con espacios ("Practica 15") causaba errores en la terminal.	Se colocaron comillas dobles alrededor de todas las rutas con espacios, o se abrió la terminal directamente desde VSCode con la carpeta del proyecto activa.
La variable <code>PICO_SDK_PATH</code> no reconocida en la terminal después de <code>setx</code> .	Se cerró y volvió a abrir la terminal para que la variable de entorno definida con <code>setx</code> tomara efecto en la nueva sesión.

3. Preguntas de Repaso

1. ¿Cuál es la principal diferencia entre Bluetooth clásico y Bluetooth LE?

La diferencia fundamental radica en el consumo energético y el caso de uso para el que fueron diseñados. El **Bluetooth clásico (BR/EDR)** está optimizado para transferencia continua de datos a velocidades de hasta 3 Mb/s (por ejemplo, audio estéreo o archivos), operando sobre 79 canales con una potencia de transmisión de hasta 100 mW. El **Bluetooth LE**, en cambio, fue diseñado para comunicaciones intermitentes con un consumo de energía drásticamente menor (≤ 10 mW), operando sobre 40 canales y tasas de hasta 2 Mb/s. Esta diferencia lo hace ideal para sensores y dispositivos alimentados

por baterías pequeñas que deben operar durante meses o años sin recarga. Adicionalmente, BLE soporta topologías de malla (Bluetooth 5) y funciones de posicionamiento mediante RSSI y ángulo de llegada, características ausentes en Bluetooth clásico.

2. ¿Qué ventajas ofrece Bluetooth LE en comparación con otras tecnologías inalámbricas?

Comparado con otras tecnologías de comunicación inalámbrica, Bluetooth LE presenta varias ventajas relevantes:

- **Frente a WiFi:** BLE consume entre 10 y 100 veces menos energía, no requiere infraestructura de red (router/AP) y tiene menor latencia de conexión inicial. WiFi es superior en ancho de banda pero impráctica para nodos de sensor con baterías pequeñas.
- **Frente a Zigbee:** BLE está integrado de forma nativa en prácticamente todos los smartphones modernos, eliminando la necesidad de gateways o coordinadores especializados para interactuar con dispositivos de usuario final.
- **Frente a NFC:** BLE opera a distancias de hasta 100 m (vs. los pocos centímetros de NFC), permitiendo monitoreo remoto sin contacto físico.
- **Frente a comunicación por cable:** BLE elimina el costo y la complejidad del cableado físico, permite movilidad del nodo sensor y facilita el escalado de la red sin modificaciones de infraestructura.

3. ¿Cuál es el propósito del archivo `btstack_config.h` en esta práctica?

El archivo `btstack_config.h` es el punto central de configuración en tiempo de compilación de la biblioteca BTstack. Su propósito es triple:

1. **Habilitar roles y características:** mediante directivas `#define ENABLE_*` se activan funcionalidades específicas como `ENABLE_LE_PERIPHERAL` (servidor) o `ENABLE_LE_CENTRAL` (cliente). En esta práctica, la directiva `RUNNING_AS_CLIENT` controla condicionalmente cuál rol se compila.
2. **Configurar la memoria estática:** define el tamaño máximo de buffers ACL (`HCI_ACL_PAYLOAD_SIZE`), el número máximo de conexiones (`MAX_NR_HCI_CONNECTIONS`), el tamaño de la base de datos ATT (`MAX_ATT_DB_SIZE`) y otros límites de memoria que determinan el footprint del stack en RAM.
3. **Configurar el control de flujo:** habilita mecanismos como `ENABLE_HCI_CONTROLLER_TO_HOST_FLOW` para evitar desbordamientos en el bus compartido del chip CYW43 de la Pico W.

4. ¿Qué ventajas tiene usar comunicación BLE en un sistema de monitoreo frente a WiFi o comunicación por cable?

En un sistema de monitoreo de temperatura como el implementado en esta práctica, BLE ofrece ventajas concretas:

- **Autonomía energética:** un nodo sensor BLE puede operar con una batería de botón CR2032 durante meses, mientras que un módulo WiFi activo consume entre 10 y 50 veces más corriente durante la transmisión.
- **Despliegue rápido:** no requiere configuración de red, credenciales WiFi ni infraestructura. El servidor simplemente anuncia su presencia y el cliente se conecta automáticamente.
- **Escalabilidad:** múltiples sensores BLE pueden operar simultáneamente en el mismo espacio sin requerir direccionamiento IP ni configuración de router.
- **Movilidad:** el sensor puede moverse libremente dentro del rango BLE (hasta 100 m en campo abierto) sin necesidad de reconfigurarse.
- **Frente al cable:** elimina el costo de instalación, el riesgo de rotura mecánica y permite monitorear ubicaciones de difícil acceso.

5. ¿Por qué es importante que el cliente BLE esté diseñado para reconectarse automáticamente al servidor en caso de desconexión?

La reconexión automática es un requisito crítico en sistemas de monitoreo por varias razones:

- **Robustez operativa:** en entornos reales, las desconexiones son inevitables por interferencias de radio, obstáculos físicos o reinicios del servidor. Un sistema que no se reconecta requiere intervención manual, lo que es inaceptable en monitoreo continuo.
- **Continuidad de datos:** en aplicaciones de monitoreo de temperatura, una desconexión sin reconexión automática puede crear brechas en los datos registrados, comprometiendo la integridad del historial.
- **Experiencia del usuario:** en el código de esta práctica, al recibir el evento `HCI_EVENT_DISCONNECTED`, el cliente llama inmediatamente a `client_start()` para reiniciar el escaneo, garantizando que el sistema se recupere sin intervención humana y que el LED indique correctamente el estado de conexión.
- **Tolerancia a fallos:** en sistemas embebidos sin pantalla ni interfaz de usuario, la reconexión automática es el único mecanismo de recuperación disponible ante fallos transitorios de comunicación.

6. En un ambiente con muchos dispositivos BLE, ¿qué estrategias se podrían usar para evitar interferencias o confusiones al conectar dispositivos?

En entornos densos con múltiples dispositivos BLE, existen varias estrategias complementarias:

- **Filtrado por UUID de servicio:** como se implementa en esta práctica, el cliente filtra los anuncios BLE buscando específicamente el UUID del servicio `Environmental Sensing`. Esto descarta todos los dispositivos que no ofrezcan ese servicio específico.
- **Filtrado por dirección MAC:** el cliente puede configurarse para conectarse únicamente a la dirección MAC conocida del servidor, almacenada previamente durante un proceso de emparejamiento inicial.
- **Nombre de dispositivo personalizado:** el servidor puede anunciar un nombre único (como "Sensor_Lab_01") que el cliente verifica antes de intentar la conexión.
- **Lista blanca (whitelist):** BTstack soporta listas blancas de dispositivos confiables mediante `gap_whitelist_add()`, permitiendo que el controlador BLE filtre a nivel de hardware.
- **Canales de anuncio escalonados:** distribuir los intervalos de anuncio entre dispositivos reduce las colisiones en los canales de anuncio (37, 38, 39).
- **Emparejamiento y bonding:** almacenar las claves de enlace permite reconocerse mutuamente sin necesidad de redescubrimiento en cada conexión.

7. ¿Cómo modificarías esta práctica para mostrar los datos en una aplicación móvil usando Bluetooth LE?

Para mostrar los datos de temperatura en una aplicación móvil se realizarían las siguientes modificaciones:

1. **En el servidor (sin cambios mayores):** el servidor ya implementa el perfil GATT estándar con el servicio `Environmental Sensing` y la característica de temperatura con soporte de notificaciones. Un smartphone puede conectarse directamente a él sin modificar el código del servidor, ya que este perfil es reconocido por el sistema operativo móvil.
2. **Aplicación Android con Jetpack Compose:** se implementaría una Activity que utiliza el `BluetoothLeScanner` del API de Android para escanear dispositivos con el UUID de servicio `0x181A` (`Environmental Sensing`), se conecta mediante `BluetoothGatt`, descubre los servicios y se suscribe a las notificaciones de la característica de temperatura `0x2A6E`. Los valores recibidos se muestran en tiempo real en la interfaz.

3. **Aplicación con MIT App Inventor o Flutter:** para un desarrollo más rápido, herramientas como MIT App Inventor (con el componente BluetoothLE) o Flutter con el paquete `flutter_blue_plus` permiten crear la interfaz sin conocimientos profundos de programación móvil nativa.
4. **Aplicación genérica BLE:** aplicaciones como *nRF Connect* (Nordic Semiconductor) o *LightBlue* ya permiten conectarse al servidor de esta práctica sin desarrollo adicional, visualizando el valor de la característica de temperatura en tiempo real.
5. **Consideraciones de formato:** el valor de temperatura se transmite como un entero de 16 bits con signo en formato little-endian, representando la temperatura en centésimas de grado Celsius. La aplicación móvil debe dividir el valor recibido entre 100 para obtener los grados Celsius, replicando la lógica del `client.c`.

4. Conclusiones

Esta práctica permitió adquirir conocimiento práctico sobre el protocolo Bluetooth Low Energy aplicado a un caso de uso real de monitoreo de sensores con microcontroladores. La implementación exitosa de la comunicación BLE entre dos Raspberry Pi Pico W mediante el perfil GATT demostró la viabilidad de desplegar sistemas de sensores inalámbricos de bajo consumo con el Pico SDK y la biblioteca BTstack.

Uno de los aprendizajes más relevantes fue comprender el **modelo de programación basado en eventos** de BTstack: a diferencia de los bucles de polling bloqueantes, toda la lógica de comunicación se estructura como máquinas de estados que reaccionan a eventos asíncronos. Este paradigma, aunque inicialmente más complejo de entender, es significativamente más eficiente en sistemas embebidos y es el mismo que se utiliza en stacks Bluetooth de nivel industrial.

El proceso de configuración del entorno de compilación en Windows presentó retos adicionales (conflictos entre compiladores, submódulos no inicializados, variables de entorno) que reforzaron habilidades fundamentales en desarrollo embebido: diagnóstico de errores de toolchain, gestión de dependencias con Git y comprensión de la cadena de compilación cruzada para arquitecturas ARM.

La reconexión automática implementada en el cliente BLE es un ejemplo de diseño robusto: el sistema se recupera de desconexiones transitorias sin intervención humana, un requisito indispensable en cualquier sistema de monitoreo desplegado en campo.

Posibles mejoras:

- **Múltiples características:** extender el servidor para exponer también humedad, presión u otros parámetros ambientales dentro del mismo servicio Environmental Sensing.
- **Almacenamiento de datos:** implementar un buffer circular en el cliente que almacene las últimas N lecturas en la flash de la Pico W usando la API de NVM del SDK.

- **Dashboard web:** conectar el cliente a WiFi (modo dual BLE+WiFi del CYW43) para reenviar los datos recibidos a un servidor MQTT o una API REST, habilitando visualización desde cualquier navegador.
- **Seguridad:** implementar emparejamiento con autenticación (*bonding*) para evitar que dispositivos no autorizados se conecten al servidor de temperatura.
- **Modo de bajo consumo:** configurar el servidor en modo de bajo consumo (*sleep*) entre mediciones, reduciendo el consumo a los pocos μA típicos de BLE en reposo.
- **Red de sensores:** escalar la arquitectura para que un único cliente pueda monitorear múltiples servidores BLE simultáneamente, creando una red de sensores distribuida.

Anexos

Anexo A – Archivo temp_sensor.gatt

```
1 PRIMARY_SERVICE, GAP_SERVICE
2 CHARACTERISTIC, GAP_DEVICE_NAME, READ, "picow temp"
3
4 PRIMARY_SERVICE, GATT_SERVICE
5 CHARACTERISTIC, GATT_DATABASE_HASH, READ,
6
7 PRIMARY_SERVICE, ORG_BLUETOOTH_SERVICE_ENVIRONMENTAL_SENSING
8 CHARACTERISTIC, ORG_BLUETOOTH_CHARACTERISTIC_TEMPERATURE,
9     READ | NOTIFY | INDICATE | DYNAMIC,
```

Anexo B – Archivo btstack_config.h

```
1 #ifndef PICO_BTSTACK_BTSTACK_CONFIG_H
2 #define PICO_BTSTACK_BTSTACK_CONFIG_H
3
4 #ifndef ENABLE_BLE
5 #error Please link to pico_btstack_ble
6 #endif
7
8 #define ENABLE_LE_PERIPHERAL
9 #define ENABLE_LOG_INFO
10 #define ENABLE_LOG_ERROR
11 #define ENABLE_PRINTF_HEXDUMP
12
13 #if RUNNING_AS_CLIENT
14 #define ENABLE_LE_CENTRAL
15 #define MAX_NR_GATT_CLIENTS 1
16 #else
```

```
17 #define MAX_NR_GATT_CLIENTS 0
18 #endif
19
20 #define HCI_OUTGOING_PRE_BUFFER_SIZE 4
21 #define HCI_ACL_PAYLOAD_SIZE (255 + 4)
22 #define HCI_ACL_CHUNK_SIZE_ALIGNMENT 4
23 #define MAX_NR_HCI_CONNECTIONS 1
24 #define MAX_NR_SM_LOOKUP_ENTRIES 3
25 #define MAX_NR_WHITELIST_ENTRIES 16
26 #define MAX_NR_LE_DEVICE_DB_ENTRIES 16
27
28 #define MAX_NR_CONTROLLER_ACL_BUFFERS 3
29 #define MAX_NR_CONTROLLER_SCO_PACKETS 3
30
31 #define ENABLE_HCI_CONTROLLER_TO_HOST_FLOW_CONTROL
32 #define HCI_HOST_ACL_PACKET_LEN (255+4)
33 #define HCI_HOST_ACL_PACKET_NUM 3
34 #define HCI_HOST_SCO_PACKET_LEN 120
35 #define HCI_HOST_SCO_PACKET_NUM 3
36
37 #define NVM_NUM_DEVICE_DB_ENTRIES 16
38 #define NVM_NUM_LINK_KEYS 16
39 #define MAX_ATT_DB_SIZE 512
40
41 #define HAVE_EMBEDDED_TIME_MS
42 #define HAVE_ASSERT
43 #define HCI_RESET_RESEND_TIMEOUT_MS 1000
44 #define ENABLE_SOFTWARE_AES128
45 #define ENABLE_MICRO_ECC_FOR_LE_SECURE_CONNECTIONS
46
47 #endif
```

Anexo C – Archivo server_common.c

```
1 #include <stdio.h>
2 #include "btstack.h"
3 #include "hardware/adc.h"
4 #include "temp_sensor.h"
5 #include "server_common.h"
6
7 #define APP_AD_FLAGS 0x06
8 static uint8_t adv_data[] = {
9     0x02, BLUETOOTH_DATA_TYPE_FLAGS, APP_AD_FLAGS,
10     0x17, BLUETOOTH_DATA_TYPE_COMPLETE_LOCAL_NAME,
11     'P', 'i', 'c', 'o', ' ', '0', '0', ':', '0', '0', ':',
12     '0', '0', ':', '0', '0', ':', '0', '0', ':', '0', '0',
```

```
13     0x03,  
        BLUETOOTH_DATA_TYPE_COMPLETE_LIST_OF_16_BIT_SERVICE_CLASS_UUIDS  
14     ,  
        0x1a, 0x18,  
15 };  
16 static const uint8_t adv_data_len = sizeof(adv_data);  
17  
18 int le_notification_enabled;  
19 hci_con_handle_t con_handle;  
20 uint16_t current_temp;  
21  
22 void packet_handler(uint8_t packet_type, uint16_t channel,  
23                     uint8_t *packet, uint16_t size) {  
24     UNUSED(size); UNUSED(channel);  
25     bd_addr_t local_addr;  
26     if (packet_type != HCI_EVENT_PACKET) return;  
27     uint8_t event_type = hci_event_packet_get_type(packet);  
28     switch (event_type) {  
29         case BTSTACK_EVENT_STATE:  
30             if (btstack_event_state_get_state(packet) !=  
31                 HCI_STATE_WORKING) return;  
32             gap_local_bd_addr(local_addr);  
33             printf("BTstack up and running on %s.\n",  
34                 bd_addr_to_str(local_addr));  
35             uint16_t adv_int_min = 800, adv_int_max = 800;  
36             uint8_t adv_type = 0;  
37             bd_addr_t null_addr;  
38             memset(null_addr, 0, 6);  
39             gap_advertisements_set_params(adv_int_min,  
40                 adv_int_max,  
41                 adv_type, 0, null_addr, 0x07, 0x00);  
42             gap_advertisements_set_data(adv_data_len, (uint8_t  
43                 *)adv_data);  
44             gap_advertisements_enable(1);  
45             poll_temp();  
46             break;  
47         case HCI_EVENT_DISCONNECTION_COMPLETE:  
48             le_notification_enabled = 0;  
49             break;  
50         case ATT_EVENT_CAN_SEND_NOW:  
51             att_server_notify(con_handle,  
52                 ATT_CHARACTERISTIC_ORG_BLUETOOTH_CHARACTERISTIC_TEMPERATURE_  
53                 ,  
54                 (uint8_t *)&current_temp, sizeof(current_temp)  
55                 );  
56             break;  
57         default: break;  
58     }
```



```
53 }
54
55 uint16_t att_read_callback(hci_con_handle_t connection_handle,
56     uint16_t att_handle, uint16_t offset,
57     uint8_t *buffer, uint16_t buffer_size) {
58     UNUSED(connection_handle);
59     if (att_handle ==
60         ATT_CHARACTERISTIC_ORG_BLUETOOTH_CHARACTERISTIC_TEMPERATURE_01_VALUE_1
61     )
62         return att_read_callback_handle_blob((const uint8_t *)
63             &current_temp,
64             sizeof(current_temp), offset, buffer, buffer_size)
65         ;
66     return 0;
67 }
68
69 int att_write_callback(hci_con_handle_t connection_handle,
70     uint16_t att_handle, uint16_t transaction_mode,
71     uint16_t offset, uint8_t *buffer, uint16_t buffer_size) {
72     UNUSED(transaction_mode); UNUSED(offset); UNUSED(
73         buffer_size);
74     if (att_handle !=
75         ATT_CHARACTERISTIC_ORG_BLUETOOTH_CHARACTERISTIC_TEMPERATURE_01_CLIENT_CHARACTERISTIC
76     )
77         return 0;
78     le_notification_enabled = little_endian_read_16(buffer, 0)
79     ==
80     GATT_CLIENT_CHARACTERISTICS_CONFIGURATION_NOTIFICATION
81     ;
82     con_handle = connection_handle;
83     if (le_notification_enabled)
84         att_server_request_can_send_now_event(con_handle);
85     return 0;
86 }
87
88 void poll_temp(void) {
89     adc_select_input(ADC_CHANNEL_TEMPSENSOR);
90     uint32_t raw32 = adc_read();
91     const uint32_t bits = 12;
92     uint16_t raw16 = raw32 << (16 - bits) | raw32 >> (2 * bits
93         - 16);
94     const float conversion_factor = 3.3f / (65535);
95     float reading = raw16 * conversion_factor;
96     float deg_c = 27.0f - (reading - 0.706f) / 0.001721f;
97     current_temp = (uint16_t)(deg_c * 100);
98     printf("Write temp %.2f degc\n", deg_c);
99 }
```

Anexo D – Archivo CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.12)
2 include(pico_sdk_import.cmake)
3
4 if (PICO_SDK_VERSION_STRING VERSION_LESS "1.5.0")
5     message(FATAL_ERROR "Pico SDK 1.5.0 or later required.")
6 endif()
7
8 project(Practica_15 C CXX ASM)
9 set(PICO_BOARD pico_w)
10 set(CMAKE_C_STANDARD 11)
11 set(CMAKE_CXX_STANDARD 17)
12 pico_sdk_init()
13
14 # Servidor (RPi Pico 1)
15 add_executable(picow_ble_temp_sensor server.c server_common.c)
16 target_link_libraries(picow_ble_temp_sensor
17     pico_stdlib pico_btstack_ble pico_btstack_cyw43
18     pico_cyw43_arch_none hardware_adc)
19 target_include_directories(picow_ble_temp_sensor PRIVATE
20     ${CMAKE_CURRENT_LIST_DIR})
21 pico_btstack_make_gatt_header(picow_ble_temp_sensor PRIVATE
22     "${CMAKE_CURRENT_LIST_DIR}/temp_sensor.gatt")
23 pico_add_extra_outputs(picow_ble_temp_sensor)
24
25 # Cliente (RPi Pico 2)
26 add_executable(picow_ble_temp_reader client.c)
27 target_link_libraries(picow_ble_temp_reader
28     pico_stdlib pico_btstack_ble pico_btstack_cyw43
29     pico_cyw43_arch_none hardware_adc)
30 pico_enable_stdio_usb(picow_ble_temp_reader 1)
31 pico_enable_stdio_uart(picow_ble_temp_reader 0)
32 target_include_directories(picow_ble_temp_reader PRIVATE
33     ${CMAKE_CURRENT_LIST_DIR})
34 target_compile_definitions(picow_ble_temp_reader PRIVATE
35     RUNNING_AS_CLIENT=1)
36 pico_add_extra_outputs(picow_ble_temp_reader)
```

Evidencia Fotográfica

En esta sección se presentan capturas del monitor serial mostrando el sistema en funcionamiento, con el cliente BLE recibiendo datos de temperatura del servidor y recuperándose automáticamente de una desconexión.

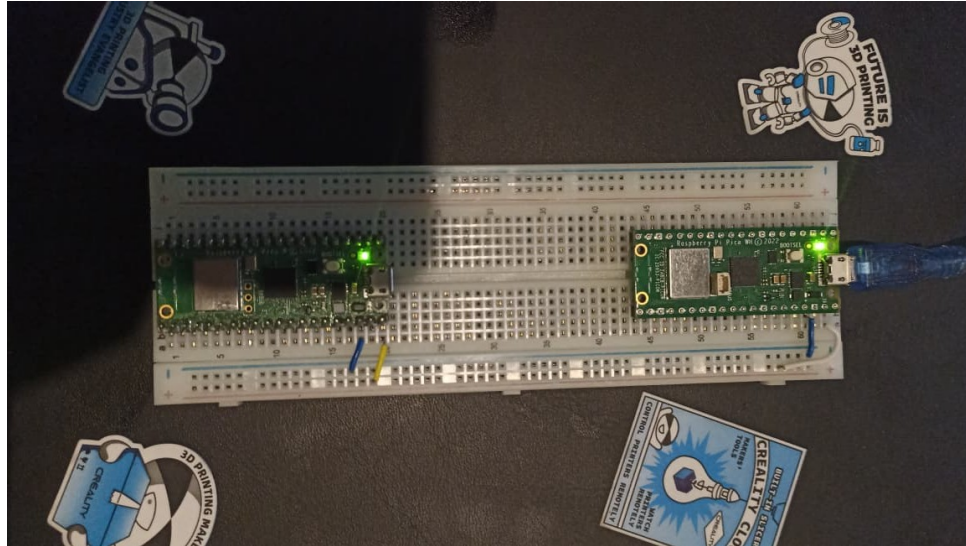


Figura 1: Vista general del montaje: dos Raspberry Pi Pico W, la de la izquierda actúa como servidor BLE (sensor de temperatura) y la de la derecha como cliente BLE (lector conectado al monitor serial).

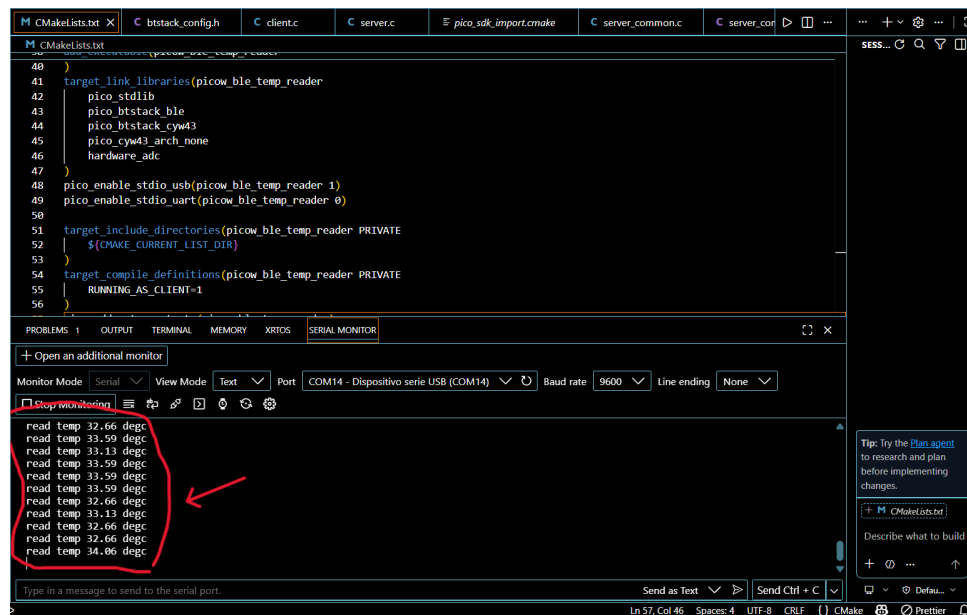


Figura 2: Monitor serial del cliente BLE mostrando lecturas de temperatura recibidas del servidor vía BLE, incluyendo un evento de desconexión y reconexión automática.

Referencias

- [1] Raspberry Pi Ltd., *Raspberry Pi Pico C/C++ SDK* (v2.2). <https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf>, 2024.

- [2] Raspberry Pi Ltd., *Raspberry Pi Pico W Datasheet*. <https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf>, 2024.
- [3] BlueKitchen GmbH, *BTstack Manual*. <https://bluekitchen-gmbh.com/btstack/>, 2024.
- [4] Bluetooth SIG, *Bluetooth Core Specification v5.4*. <https://www.bluetooth.com/specifications/specs/core-specification/>, 2023.
- [5] Bluetooth SIG, *GATT Specification Supplement*. <https://www.bluetooth.com/specifications/specs/gatt-specification-supplement/>, 2023.
- [6] Raspberry Pi Ltd., *RP2040 Datasheet*. <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>, 2024.